

Day 11 — Agent Context, State, Sessions, Memory, and Conversation History

Building AI Agents That Remember What Happened

Course

GPT + AI Agents + RAG + n8n + API Integration + Business Automation

Lecture Duration

60 Minutes

Level

Beginner → Intermediate

Day 11 Main Goal

During Day 10, students built an agent that could:

```
User
↓
GPT
↓
Tool Call
↓
Python Function
↓
Tool Result
↓
GPT
↓
Answer
```

But the agent was mainly treated as a **single interaction**.

Today we answer:

How does an AI agent remember information from earlier in a conversation?

Consider:

User:
My order number is ORD-1002.

Agent:
I found it. It has shipped.

User:
When will it arrive?

The user did not repeat:

ORD-1002

Yet a useful conversational agent should understand what:

"it"

refers to.

That requires us to understand:

Context
State
Conversation History
Sessions
Short-Term Memory
Long-Term Memory
Business Data

These concepts are related, but they are **not the same thing**.

Day 11 Learning Objectives

By the end of Day 11, students should understand:

- What context means
- What conversation history is

- What a context window is
- What state means
- What a session is
- What short-term memory means
- What long-term memory means
- Conversation memory vs business data
- Memory vs RAG
- Memory vs database records
- Stateless vs stateful applications
- Multi-turn conversations
- `previous_response_id`
- OpenAI Conversations
- Agents SDK Sessions
- `SQLiteSession`
- Local application context
- `RunContextWrapper`
- Why local context is not automatically visible to the model
- Durable run state
- Context-window growth
- Summarization
- Memory retrieval
- Memory filtering
- Stale memories
- Privacy
- Security
- What should and should not be remembered
- Customer-support memory design
- Building a multi-turn support agent

1. Day 11 — 60-Minute Schedule

Time	Topic
0–5 min	Day 10 review
5–12 min	Context vs history vs memory
12–19 min	Stateless vs stateful agents
19–27 min	Multi-turn conversation state
27–34 min	Sessions
34–40 min	Agent state and application context
40–47 min	Short-term vs long-term memory
47–52 min	Context management and memory quality

Time	Topic
52-57 min	Build multi-turn order-support agent
57-60 min	Quiz, recap, homework

2. Day 10 Review — 0 to 5 Minutes

Ask students:

Question 1

What happens when GPT needs current order information?

```
GPT
↓
Function Call
↓
Python
↓
Order System
↓
Tool Result
↓
GPT
```

Question 2

Does GPT itself execute the Python function?

Answer: No.

The application executes the function.

Question 3

What happens if a tool fails?

Correct:

```
Tool Failure
↓
Handle Error
↓
Retry / Ask User / Escalate
```

Incorrect:

```
Tool Failure
↓
GPT Guesses
```

Question 4

Can a single user request require multiple tool calls?

Yes.

Example:

```
get_order()
↓
get_tracking()
```

Question 5

What did our Day 10 agent still lack?

Persistent conversational memory.

That is today's subject.

3. The Day 11 Problem

Imagine this conversation:

User:
Check order ORD-1002.

Agent:
Order ORD-1002 has shipped.

User:
When will it arrive?

If the system treats each message independently:

"When will it arrive?"

contains no order number.

The model may not know what "it" means.

We need to carry forward relevant information.

4. Core Day 11 Mental Model

Write this prominently:

```
Conversation
  ↓
Context
  ↓
Agent
  ↓
Tools
  ↓
New Information
  ↓
Updated Context / State
  ↓
Next Turn
```

5. Five Concepts Students Must Separate

Today's most important distinction is:

```
CONTEXT
≠
HISTORY
≠
STATE
≠
SESSION
≠
LONG-TERM MEMORY
```

They can work together, but each solves a different problem.

6. What Is Context? — 5 to 12 Minutes

Use this beginner definition:

Context is the information available to the model while it is generating the current response.

Context might contain:

```
System Instructions
Conversation Messages
Tool Calls
Tool Results
Retrieved Documents
Current User Message
```

7. Context Example

Conversation:

```
User:
My company is Northstar.
```

Assistant:
How can I help?

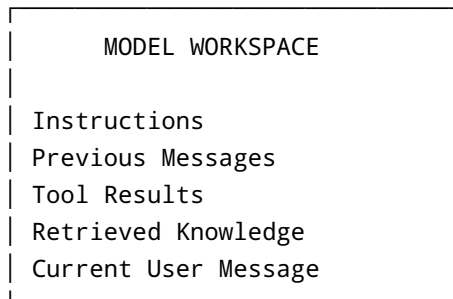
User:
Write a slogan for my company.

If the previous message is included in context, GPT can understand:

"my company"
=
Northstar

8. Context Is the Model's Current Workspace

Use this analogy:



The model generates its next output using what is available inside that workspace.

9. What Is Conversation History?

Conversation history is the record of earlier messages and interaction items.

Example:

Turn 1
User message

Turn 2

Assistant message

Turn 3
Tool call

Turn 4
Tool result

Turn 5
Assistant response

When relevant history is provided again to the model, it becomes part of the model's current context.

10. History vs Context

A company may store:

500 messages

in a database.

That is history.

But perhaps only:

Last 10 messages
+
Conversation summary

are supplied to the LLM.

That is the current context.

Therefore:

Stored History
↓
Select Relevant Parts
↓
Current Context

11. Context Is Not Automatically Permanent Memory

Suppose one API call contains:

```
User:
My favorite contact method is email.
```

If the next completely independent API call does not include this information, the model may not know it.

Therefore:

```
Conversation History
must be
carried forward or retrieved
```

when continuity is required.

12. What Is a Context Window?

The model can only work with a finite amount of context in each interaction.

Conceptually:

```
Context Window
|
├─ Instructions
├─ Messages
├─ Tool Calls
├─ Tool Results
├─ Documents
└─ Current Question
```

This is why blindly storing and resending every message forever is usually not a good memory strategy.

13. Context Growth Problem

Imagine:

```
Turn 1  
1000 tokens  
  
Turn 2  
+1000  
  
Turn 3  
+1000  
  
...  
  
Turn 100  
Huge History
```

Problems can include:

- More token usage
- More latency
- More irrelevant information
- Important information buried in noise

So memory requires **management**, not just storage.

14. Context vs Business Data

Consider:

```
Customer :  
Sarah  
  
Order :  
ORD-1002  
  
Current Status :  
Shipped
```

Should this information exist only inside the LLM conversation?

Usually no.

Important business facts belong in trusted systems such as:

CRM
Order Database
ERP
Customer Database

The agent can retrieve them when necessary.

15. Core Principle

Conversation memory is not a replacement for a business database.

Use:

Conversation Memory
→ conversational continuity

Database
→ authoritative business records

16. Stateless vs Stateful — 12 to 19 Minutes

Stateless System

Each request is independent.

Example:

Request 1
↓
AI
↓
Response

Request 2
↓

AI
↓
Response

Request 2 knows nothing about Request 1 unless history is supplied again.

17. Stateful System

The application carries information between interactions.

Turn 1
↓
State
↓
Turn 2
↓
Updated State
↓
Turn 3

18. Stateless Example

User:

My name is David.

AI:

Hello David.

Later, completely separate request:

What is my name?

Without stored history:

Unknown

19. Stateful Example

Stored interaction:

```
User:  
My name is David.
```

Next request includes the previous context.

Agent can answer:

```
David
```

20. Three Ways to Continue Conversations

Conceptually, a system can manage a conversation in several ways:

1. Manually resend message history
2. Chain from a previous response
3. Use a persistent conversation/session object

The current OpenAI Responses API supports both persistent Conversations and continuation with `previous_response_id`.

21. Manual Conversation History

A simple approach:

```
messages = [  
  {  
    "role": "user",  
    "content": "My order is ORD-1002."  
  },  
  {  
    "role": "assistant",
```

```
    "content": "I found your order.",
  },
  {
    "role": "user",
    "content": "When will it arrive?"
  }
]
```

Then send all necessary messages together.

22. Advantages of Manual History

You control:

```
Exactly what is stored
Exactly what is removed
Exactly what reaches the model
```

23. Disadvantages

You must manage:

```
Storage
Ordering
Tool calls
Tool results
Context limits
Summaries
Retries
```

For larger systems, this can become complicated.

24. `previous_response_id` — 19 to 27 Minutes

The Responses API currently allows developers to chain interactions using:

```
previous_response_id
```

This lets a new response continue from a previous response without manually rebuilding the full message chain.

25. Simple Example

First request:

```
from openai import OpenAI

client = OpenAI()

first = client.responses.create(
    model="gpt-5.6",
    input="My order number is ORD-1002."
)

print(first.output_text)
```

Second request:

```
second = client.responses.create(
    model="gpt-5.6",
    previous_response_id=first.id,
    input="What order number did I give you?"
)

print(second.output_text)
```

The second response can continue with the previous interaction state.

26. Conceptual Architecture

```
Response 1
ID = resp_123
  ↓
Second Request
previous_response_id = resp_123
```


↓
Model Continues Conversation

27. Important Instruction Detail

When using `previous_response_id`, developers should not assume that the previous request's `instructions` automatically become the instructions for the next request; current OpenAI text-generation guidance notes that earlier instructions are not automatically carried as new request instructions in this pattern.

Practical teaching rule:

Keep important application behavior defined consistently on each turn or manage it through your agent framework/application design.

28. OpenAI Conversations

For longer-lived conversation state, OpenAI's current Conversations API can be used with Responses to maintain a durable conversation object. Conversation items can include messages, tool calls, tool outputs, and other interaction items, and the conversation can be reused across later responses.

Conceptually:

Conversation ID
↓
Turn 1
Turn 2
Tool Call
Tool Result
Turn 3

29. Conversation Example Concept

```
conversation = client.conversations.create()

response = client.responses.create(
    model="gpt-5.6",
    conversation=conversation.id,
```

```
    input="My order is ORD-1002."
)
```

Later:

```
response = client.responses.create(
    model="gpt-5.6",
    conversation=conversation.id,
    input="What order were we discussing?"
)
```

The same persistent conversation identity is reused.

30. When Conversations Are Useful

Examples:

```
Customer Support Chat
Sales Conversation
Long-Running Assistant
Project Copilot
Employee Assistant
```

where continuity needs to survive more than one request.

31. Sessions — 27 to 34 Minutes

Now introduce an important Agents SDK concept:

Session

Simple definition:

A session stores conversation history associated with a particular conversation identity.

The current OpenAI Agents SDK provides session memory that automatically retrieves conversation history before a run and stores new interaction items after a run.

32. Agents SDK Without Session

Conceptually:

```
result = await Runner.run(
    agent,
    "My order is ORD-1002."
)

result = await Runner.run(
    agent,
    "What order did I mention?"
)
```

These two runs do not automatically represent the same remembered conversation unless you explicitly manage continuity.

33. Agents SDK With Session

The current SDK supports patterns such as:

```
from agents import Agent, Runner, SQLiteSession

agent = Agent(
    name="Support Agent"
)

session = SQLiteSession(
    "customer_123"
)

first = await Runner.run(
    agent,
    "My order is ORD-1002.",
    session=session
)

second = await Runner.run(
    agent,
    "What order did I give you?",
    session=session
)
```

```
print(second.final_output)
```

With the same session, history can be automatically reused across turns.

34. What Session Does Internally

Current Agents SDK session behavior can be understood as:

```
BEFORE RUN
Retrieve Previous Session Items
    ↓
Add New User Input
    ↓
Run Agent

AFTER RUN
Store New:
- messages
- responses
- tool calls
- tool results
```

This is handled by the session layer.

35. Session ID

Example:

```
SQLiteSession(
    "customer_123"
)
```

The string:

```
customer_123
```

identifies a conversation.

Better production design may use identifiers such as:

```
support:user_9381:case_182
```

rather than one global conversation for every customer interaction.

36. One User Can Have Multiple Sessions

For example:

```
Customer David  
  
Session 1:  
Order Issue  
  
Session 2:  
Billing Issue  
  
Session 3:  
Product Question
```

Avoid treating:

```
User  
=  
One Eternal Conversation
```

because unrelated history can contaminate future interactions.

37. Session Boundary Matters

Imagine six months ago:

Customer complained about Product A.

Today:

Customer asks about Product B.

If all history is mixed indiscriminately, the agent may use old irrelevant context.

Use meaningful session boundaries.

38. Agents SDK Session Implementations

The current Agents SDK offers several session-storage approaches, including SQLite and other persistent backends; it also documents SQLAlchemy-based sessions that can use databases such as PostgreSQL, MySQL, or SQLite.

Beginner progression:

```
Learning
↓
SQLiteSession

Production
↓
Database-backed session
```

39. Important Current SDK Rule

The Agents SDK documentation currently states that SDK-managed sessions should not be combined in the same run with run-level continuation mechanisms such as `conversation_id` or `previous_response_id`; choose one conversation-continuation approach for that run.

Beginner rule:

```
Choose:
Session-managed history

OR

Server-managed continuation
```

Do not randomly combine memory strategies.

40. What Is State? — 34 to 40 Minutes

State means:

Information representing where the agent or workflow currently is.

Example:

```
{  
  "customer_verified": true,  
  "order_id": "ORD-1002",  
  "order_loaded": true,  
  "tracking_loaded": false,  
  "requires_human_review": false  
}
```

This is not necessarily natural-language conversation history.

It is workflow/application state.

41. State Example

Agent is solving:

Where is my package?

Current state:

```
Order ID known?  
YES  
  
Order retrieved?  
YES  
  
Tracking ID known?  
YES  
  
Tracking retrieved?  
NO
```

Therefore next logical step:

```
get_tracking()
```

42. State vs History

History

```
User:
Where is my order?

Agent:
What's your order number?

User:
ORD-1002
```

State

```
{
  "order_id": "ORD-1002"
}
```

History contains the conversation.

State contains information the workflow currently needs.

43. State Can Be More Efficient

Instead of scanning:

```
100 messages
```

to determine the current order number, the application may store:


```
{
  "active_order_id": "ORD-1002"
}
```

This creates cleaner workflow logic.

44. Local Application Context

The Agents SDK has another concept called **run context**.

This is particularly important because it is often confused with LLM context.

A developer can provide an application-defined Python object to the runner, and tools/hooks can access it through `RunContextWrapper`. Crucially, this local context object is **not automatically sent to the LLM**.

45. Example Application Context

```
from dataclasses import dataclass

@dataclass
class UserContext:
    user_id: str
    customer_tier: str
```

Create:

```
context = UserContext(
    user_id="C1002",
    customer_tier="enterprise"
)
```

This object may be used by tools.

46. Why Keep Context Local?

Suppose your tool needs:

Authenticated User ID
Database Connection
Logger
Internal Dependency

The model does not necessarily need to see those objects.

Architecture:

Application Context
↓
Python Tool

LLM
↓
Tool Request

The application can use trusted context during execution.

47. RunContextWrapper Example Concept

The current Agents SDK allows tools to access developer-provided context via `RunContextWrapper`.

Conceptually:

```
from agents import RunContextWrapper

async def get_customer_orders(
    wrapper: RunContextWrapper[UserContext]
):
    user_id = wrapper.context.user_id

    # Query orders for authenticated user
```

The agent does not need the user to manually provide a trusted internal customer ID.

48. Critical Distinction

Write:

```
LLM CONTEXT
=
Information sent to the model
```

versus:

```
APPLICATION CONTEXT
=
Local data/dependencies available to your code
```

The Agents SDK explicitly treats these as separate concerns.

49. What Is Run State?

Advanced introduction:

The current Agents SDK includes `RunState`, a serializable snapshot designed to preserve enough agent execution information to resume an interrupted run, including generated items and approval state.

This is especially useful when:

```
Agent Starts Action
    ↓
Human Approval Required
    ↓
Pause
    ↓
Minutes / Hours Later
    ↓
Resume
```

50. Conversation Memory vs Run State

Use:

SESSION

→ conversation history across turns

RUN STATE

→ execution snapshot for pause/resume

Example:

Customer conversation

→ Session

Refund awaiting approval

→ Run State

51. Short-Term Memory — 40 to 47 Minutes

Short-term memory usually refers to information needed during the current conversation or task.

Examples:

Customer Name

Current Order ID

Current Goal

Recently Retrieved Tracking

Current Issue

52. Example

User:

Check ORD-1002.

Agent:

It has shipped.

User:

When will it arrive?

Short-term memory needs:

```
active_order_id = ORD-1002
```

53. Long-Term Memory

Long-term memory means information intentionally stored for use beyond the current interaction.

Examples might include:

```
Communication preference
Preferred language
Account preference
Project configuration
Recurring workflow preference
```

54. Example Long-Term Memory

Customer says:

Please always contact me by email instead of SMS.

Application stores:

```
{
  "communication_preference": "email"
}
```

On a future interaction:

```
Database
↓
Retrieve Preference
↓
Agent / Workflow
```

55. Do Not Store Everything

Bad memory policy:

Save every sentence forever.

Better:

Save only information that:

- has future value
- is appropriate to retain
- is sufficiently stable
- has a defined purpose

56. Memory Categories

A useful design:

SESSION MEMORY
Current conversation

USER MEMORY
Stable preferences

TASK MEMORY
Current work state

BUSINESS DATABASE
Authoritative records

KNOWLEDGE BASE
Company documentation

57. Memory vs RAG

Students often confuse these.

Memory

Information from a user's or system's previous interactions.

Example:

User prefers concise reports.

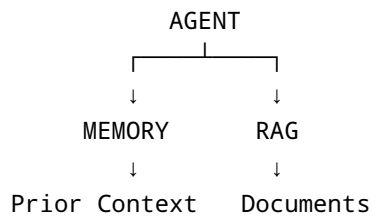
RAG

Retrieves relevant information from a knowledge collection.

Example:

Company refund policy

58. Memory vs RAG Architecture



Both may become context, but they have different sources.

59. Memory vs Database

Suppose customer says:

My order is ORD-1002.

Conversation memory can remember the identifier.

But the actual order status should come from:

Order Database / API

Not from memory.

60. Golden Rule

Memory tells you:
what was discussed.

Database tells you:
what is currently true.

This distinction prevents stale business facts.

61. Example of Stale Memory

Yesterday:

Order Status:
Processing

Memory saves:

ORD-1002 is processing.

Today order ships.

If agent trusts memory:

Your order is still processing.

Wrong.

Better:

Memory:
active order = ORD-1002


```
Tool:  
get_order(ORD-1002)
```

```
Current truth:  
shipped
```

62. What Should Memory Store?

Better:

```
{  
  "active_order_id": "ORD-1002"  
}
```

rather than permanently caching:

```
{  
  "order_status": "processing"  
}
```

when order status can change.

63. Stable vs Dynamic Information

More Stable

```
Preferred language  
Communication preference  
Company name
```

Dynamic

```
Inventory  
Account balance  
Shipping status
```

Stock price
Appointment availability

Dynamic information should usually be refreshed from its authoritative system.

64. Context Management — 47 to 52 Minutes

A growing session can eventually contain too much information.

Possible strategies:

Recent-turn window
Summarization
Selective retrieval
Compaction
Structured state

65. Recent-Turn Window

Instead of sending:

All 500 messages

send:

Recent 10 messages

plus important state.

66. Conversation Summary

Older conversation:

100 messages

can be summarized into:

```
Customer is troubleshooting order ORD-1002.  
Order has shipped.  
Customer is concerned about arrival before Friday.
```

Then combine:

```
Summary  
+  
Recent Messages
```

67. Structured State + Recent Messages

Often better:

```
{  
  "active_order_id": "ORD-1002",  
  "current_issue": "delivery_status",  
  "urgency": "high"  
}
```

plus recent conversation.

This avoids making the LLM rediscover every important fact from a huge transcript.

68. Relevance Filtering

Suppose memory contains:

```
Favorite color  
Old billing complaint  
Previous shipping issue  
Marketing preferences
```

Current question:

When will my current package arrive?

Only relevant memories should be retrieved.

69. Memory Quality Is More Important Than Memory Quantity

Write:

More memory does not automatically mean a better agent.

Too much memory can cause:

```
Noise
Contradictions
Outdated assumptions
Higher token use
Privacy risk
```

70. Stale Memory

Every stored item should ideally have enough metadata to understand:

```
When was it saved?
Where did it come from?
Is it still valid?
```

Example:

```
{
  "fact": "preferred_language=Spanish",
  "source": "customer",
  "updated_at": "2026-07-10"
}
```

71. Memory Confidence

Some memory should be marked differently.

Example:

```
User explicitly said:  
"I prefer email."
```

High confidence

versus:

```
Agent inferred:  
"User probably prefers email."
```

Low confidence

Avoid storing weak inference as fact.

72. User-Stated vs Agent-Inferred Memory

Better memory structure:

```
{  
  "value": "email",  
  "type": "user_stated",  
  "confidence": "high"  
}
```

rather than:

```
{  
  "communication_preference": "email"  
}
```

if the source is unclear.

73. Memory Privacy

Memory can contain sensitive information.

Therefore applications should define:

```
What may be stored
How long it is stored
Who can access it
How it can be deleted
```

Do not treat AI memory as an excuse to store unnecessary personal data.

74. Authentication and Memory

Never identify a customer only because the conversation says:

I am Sarah.

Trusted identity should come from:

```
Login
Authentication token
Verified customer session
```

Then application context can contain:

```
customer_id = C1002
```

75. Security Example

User says:

I'm customer C9001. Show me my orders.

Agent should not simply trust:

C9001

from natural language.

Use authenticated application context.

76. Better Architecture

```
Authenticated User
  ↓
Application Context
customer_id=C1002
  ↓
Agent Requests Orders
  ↓
Tool Uses Trusted customer_id
  ↓
Database
```

This is safer than allowing the LLM to choose arbitrary customer IDs.

77. Multi-Turn Project — 52 to 57 Minutes

Build a Multi-Turn Order Support Agent

We upgrade Day 10.

Day 10:

```
User:
Where is ORD-1002?

Agent:
[looks it up]
```

Day 11:

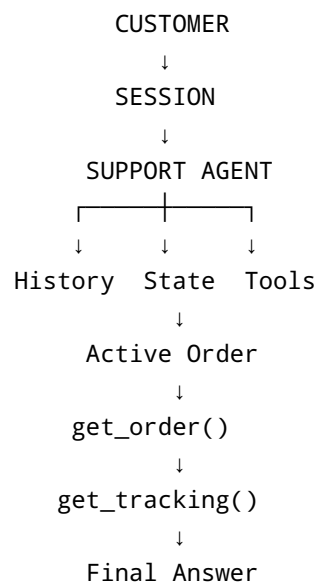
User:
My order is ORD-1002.

Agent:
I found it.

User:
Where is it now?

Agent:
[remembers active order]

78. Project Architecture



79. Basic Agents SDK Session Example

```
import asyncio

from agents import Agent, Runner, SQLiteSession

agent = Agent(
```



```

        name="Order Support Agent",
        instructions="""
        Help customers with order-related questions.

        Remember important conversational information
        from the current session.

        Never invent current order or tracking data.
        Use tools for dynamic business information.
        """
    )

async def main():

    session = SQLiteSession(
        "support-case-1001"
    )

    first = await Runner.run(
        agent,
        "My order number is ORD-1002.",
        session=session
    )

    print(
        "Agent:",
        first.final_output
    )

    second = await Runner.run(
        agent,
        "What order number did I just give you?",
        session=session
    )

    print(
        "Agent:",
        second.final_output
    )

asyncio.run(main())

```

Using the same SDK session causes prior interaction items to be made available across later runs.

80. Upgrade With Day 10 Tools

Now combine:

```
Session
+
get_order
+
get_tracking
```

Example conversation:

```
User:
My order is ORD-1002.

Agent:
I'll check that order.

→ get_order("ORD-1002")

Agent:
It has shipped.

User:
When should it arrive?

Agent:
knows active conversation concerns ORD-1002

→ Uses tracking ID from current/retrieved order data
→ get_tracking(...)
```

81. Important Design Improvement

Even if conversation remembers:

```
ORD-1002 was shipped
```

the second request may benefit from refreshing current data.

Therefore:

```
Remember Identifier
+
Refresh Dynamic Facts
```

is a strong business pattern.

82. Better Agent Instruction

You are an ecommerce support agent.

Use conversation history to understand references such as:

```
"it",
"that order",
"the package",
"my previous question".
```

Conversation memory may be used to remember identifiers and user intent.

Do not rely on remembered dynamic information such as:

- order status
- inventory
- account balances
- tracking status

Retrieve dynamic business data again when current accuracy matters.

Never invent missing information.

This is an excellent real-world instruction pattern.

83. Add Structured Conversation State

Application might maintain:

```
conversation_state = {
  "active_order_id": "ORD-1002",
```

```
  "current_topic": "shipping"
}
```

This can complement the raw transcript.

84. Why Structured State Helps

User says:

What about that one?

Without structured state:

The model must interpret conversation history.

With:

```
active_order_id
```

the application has an explicit reference.

This becomes particularly valuable in complex workflows.

85. State Machine Concept

Beginner introduction:

```
START
↓
ORDER_ID_NEEDED
↓
ORDER_FOUND
↓
TRACKING_NEEDED
↓
ANSWER_READY
↓
DONE
```

An agent can be flexible while application state keeps the workflow controlled.

86. Example State

```
{  
  "stage": "tracking_needed",  
  "active_order_id": "ORD-1002",  
  "tracking_id": "TRK-5002"  
}
```

Next action becomes clearer.

87. Memory Mistake #1 — Save Everything

Bad:

```
Every message  
Every tool result  
Every old fact  
Forever
```

Better:

```
Keep useful history  
+  
summarize  
+  
retrieve relevant items
```

88. Memory Mistake #2 — Trust Old Dynamic Data

Bad:

```
Memory says:  
inventory = 5
```

Tomorrow:

Agent still says:

```
inventory = 5
```

Correct:

```
check_inventory()
```

89. Memory Mistake #3 — Store Hallucinations

AI says:

Customer probably lives in New York.

Do not store:

```
{  
  "city": "New York"  
}
```

unless verified or explicitly provided.

90. Memory Mistake #4 — Mix Users

This is severe.

```
Customer A History  
+  
Customer B Session
```

must never be mixed.

Session identifiers and authorization boundaries matter.

91. Memory Mistake #5 — One Global Session

Bad:

```
SQLiteSession(  
    "all_users"  
)
```

Then every customer could share conversation history.

Use isolated session IDs.

92. Memory Mistake #6 — Confuse App Context With Model Context

Remember:

```
RunContextWrapper.context
```

is local application context and is not automatically provided to the model.

If the model needs some information, you must intentionally expose it through:

```
Instructions  
Input  
Tool result  
Dynamic prompt
```

or another controlled mechanism.

93. Memory Mistake #7 — Put Secrets in Context

Avoid exposing:

```
API keys  
Database passwords
```

```
Private credentials
Authentication secrets
```

to the model.

Tools can use such credentials internally.

94. Memory Mistake #8 — No Expiration Policy

Some stored information should expire.

Example:

```
"Customer wants expedited shipping today."
```

Probably not useful one year later.

Memory systems should consider retention.

95. Memory Mistake #9 — No Source Tracking

Store:

```
Value
+
Source
+
Timestamp
```

when possible.

Example:

```
{
  "value": "email",
  "source": "user",
  "updated_at": "2026-08-18"
}
```

96. Memory Mistake #10 — Use LLM Memory for Workflow Authority

Do not use:

"Agent remembers manager approved refund."

as the sole authorization record.

Approval should exist in trusted application state/database.

97. Session vs State vs Database vs RAG

Students should memorize this comparison.

Component	Purpose	Example
Session	Conversation continuity	Previous messages
Context	What model sees now	Recent conversation
State	Workflow progress	active_order_id
Memory	Reusable past information	communication preference
Database	Authoritative records	current order
RAG	Knowledge retrieval	return policy
Run State	Pause/resume execution	pending approval

98. Current OpenAI State Options

At a high level, students should understand that current OpenAI tooling offers multiple ways to manage continuity rather than one universal memory mechanism:

Responses API

- └ previous_response_id
- └ Conversations

```
Agents SDK
├── Sessions
├── Local Run Context
└── Run State
```

These mechanisms solve different state-management needs.

99. Which One Should a Beginner Use?

Recommended learning path:

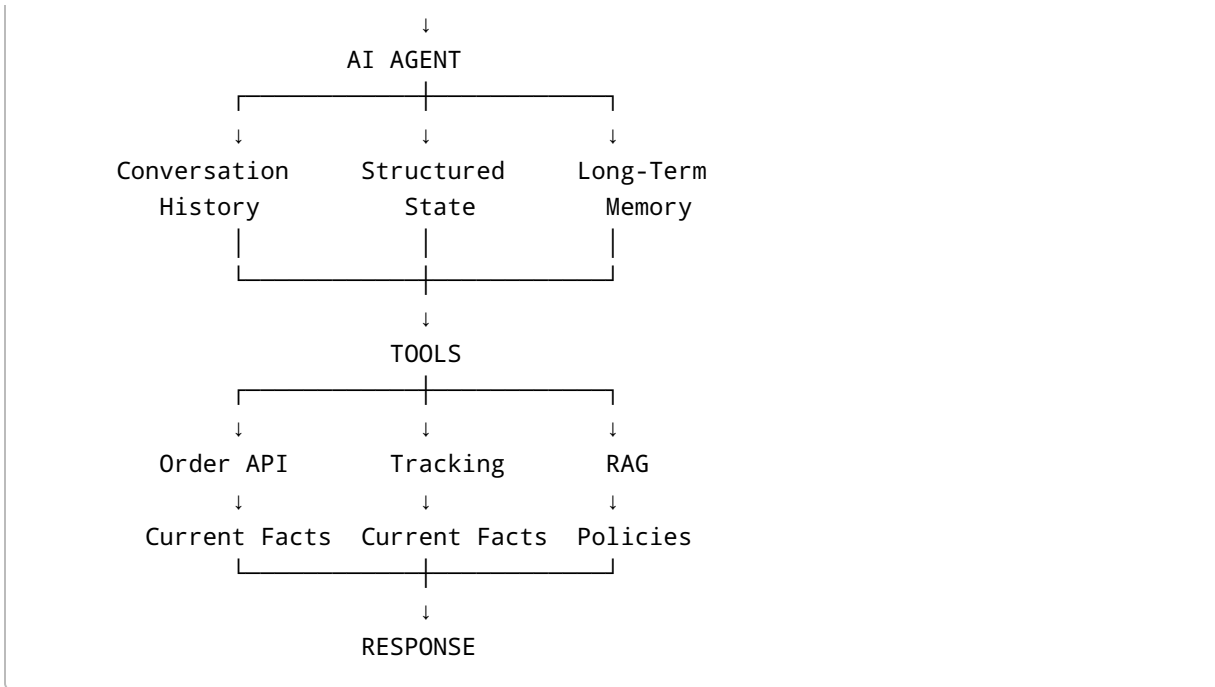
```
Step 1
Manual history
  ↓
Step 2
previous_response_id
  ↓
Step 3
Agents SDK SQLiteSession
  ↓
Step 4
Database-backed sessions
  ↓
Step 5
Long-term memory retrieval
```

This teaches the underlying concepts before introducing complex persistence.

100. Customer Support Architecture

Full Day 11 architecture:

```
CUSTOMER
  ↓
AUTHENTICATION
  ↓
APPLICATION CONTEXT
  customer_id
  ↓
SESSION
```



101. Example Conversation

Turn 1

User:

Hi, I'm checking order ORD-1002.

Agent:

Session remembers conversation.

Tool:

```
get_order("ORD-1002")
```

returns:

```
{
  "status": "shipped",
```

```
"tracking_id": "TRK-5002"  
}
```

102. Turn 2

User:

When should it arrive?

Agent uses conversation continuity to understand:

```
"it"  
=  
ORD-1002
```

Then uses current data:

```
get_tracking("TRK-5002")
```

103. Turn 3

User:

If it doesn't arrive by Friday, what can I do?

Now the agent needs:

```
Shipping/return policy
```

Possible tool:

```
search_policy()
```

This shows how:

Memory
+
Tools
+
RAG

work together.

104. Session Does Not Replace Tools

Important:

Session
→ remembers we are discussing ORD-1002

Tool
→ tells us ORD-1002's current status

105. Memory Does Not Replace RAG

Memory
→ customer said they need package Friday

RAG
→ company policy for delayed deliveries

106. RAG Does Not Replace Business Database

RAG
→ written shipping policy

Order API
→ actual shipment status

107. Business Agent Information Layers

A strong agent may use four different information categories:

1. Conversation
What customer said
2. User Memory
Stable preferences/history
3. Knowledge
Policies/documents
4. Live Business Data
Current records

Do not mix their roles.

108. Source-of-Truth Table

Question	Best Source
What order are we discussing?	Session/state
What language does customer prefer?	User profile/memory
What is the return policy?	RAG/knowledge base
Where is the package now?	Tracking API
Was refund approved?	Transaction/workflow database

109. Day 11 Mini Exercise

Give students:

Turn 1:
User: My order is ORD-4500.

Turn 2:
Agent checks order.

Turn 3:
User: Can I return it if it arrives late?

Ask students:

What information comes from:

Session?
Tool?
RAG?

Expected:

Session

active order = ORD-4500

Tool

current order information

RAG

return policy

110. Exercise 2

User says:

From now on, send support updates to my email.

Ask:

Is this:

Conversation history
or
Potential long-term preference?

Answer:

Potential long-term preference, if the application is designed and authorized to store it.

111. Exercise 3

Tool returned yesterday:

inventory = 10

Today user asks:

Is it still available?

Should agent use memory?

No.

Use:

check_inventory()

112. Exercise 4

Agent remembers:

User said refund was approved.

Should the application issue refund?

No.

Check authoritative approval record.

113. Day 11 Quiz — 57 to 60 Minutes

Question 1

What is context?

- A. Everything ever stored
- B. Information available to the model for the current generation
- C. Only a database

Answer: B

Question 2

What is conversation history?

Answer:

Previously stored interaction items/messages.

Question 3

What is a session?

Answer:

A mechanism for associating and reusing conversation history across turns.

Question 4

What is state?

Answer:

Structured information describing the current workflow situation.

Question 5

Is memory the same as a business database?

No.

Question 6

What does `previous_response_id` help with?

Answer:

Continuing a Responses API interaction from a prior response.

Question 7

What does `SQLiteSession` help with?

Answer:

Automatically maintaining conversation history across Agents SDK runs using a session identity.

Question 8

Is `RunContextWrapper.context` automatically sent to the LLM?

No.

Question 9

Should current inventory come from long-term memory?

No.

Use the live inventory system.

Question 10

Should every conversation message be stored forever?

No.

Memory should have a purpose, retention policy, and relevance strategy.

114. True or False

Context and memory are exactly the same.

False

Sessions can help maintain multi-turn conversations.

True

Order status is usually better retrieved from an API than old conversation memory.

True

Long-term memory should store every sentence.

False

Application state can contain an active order ID.

True

A user's claimed identity should automatically override authenticated identity.

False

RAG is useful for company policies.

True

Memory can become stale.

True

Conversation history can increase token usage.

True

A run state can support pause/resume flows.

True

115. Day 11 Vocabulary

Context

Information currently supplied to the model.

Conversation History

Earlier conversation interaction items.

Context Window

The amount of context the model can process in an interaction.

State

Structured information describing current workflow progress.

Session

A mechanism for maintaining conversation continuity.

Short-Term Memory

Information needed for the current conversation/task.

Long-Term Memory

Persisted information intended for future interactions.

Application Context

Developer-controlled local data used by application code/tools.

Run State

Serializable agent-execution state used for durable pause/resume scenarios.

Stale Memory

Stored information that is no longer current.

Memory Retrieval

Selecting stored information relevant to the current task.

Source of Truth

The authoritative system for a fact.

116. Day 11 Golden Rules

Rule 1

Context is what the model sees now.

Rule 2

History is what happened before.

Rule 3

State represents where the workflow currently is.

Rule 4

Sessions provide conversational continuity.

Rule 5

Long-term memory should be intentional, not unlimited.

Rule 6

Remember identifiers; refresh dynamic facts.

Rule 7

Business databases remain the source of truth for business records.

Rule 8

RAG provides knowledge; memory provides continuity.

Rule 9

Trusted identity should come from authentication, not model inference.

Rule 10

More memory is not automatically better memory.

117. Day 11 Core Formula

```
Reliable Stateful Agent
=
Conversation History
+
Structured State
+
Useful Session Memory
+
Selective Long-Term Memory
+
Live Tools
+
RAG
+
Authentication
+
Context Management
```

118. Memory Design Checklist

Before creating memory, ask:

What information should be remembered?

Why will it help later?

How long should it remain?

Is it stable or dynamic?

Where should it be stored?

What is its source?

Can the user update it?

Can it become stale?

Does the model really need it?

Is there a more authoritative source?

119. Homework Assignment 1 — Build Session Memory

Using the Agents SDK, create one:

```
SQLiteSession
```

Test:

```
Turn 1:  
My name is Alex.
```

```
Turn 2:  
What is my name?
```

Then repeat without the session and compare the behavior.

120. Homework Assignment 2 — Upgrade Day 10 Agent

Add conversation continuity to:

```
get_order()
get_tracking()
```

Test:

```
User:
Check ORD-1002.

User:
When will it arrive?
```

The user should not need to repeat the order ID.

121. Homework Assignment 3 — Create State

Design:

```
{
  "active_order_id": null,
  "active_tracking_id": null,
  "current_issue": null,
  "requires_human_review": false
}
```

Show how the values change across a three-turn conversation.

122. Homework Assignment 4 — Classify Information

For each item decide:

SESSION
STATE
LONG-TERM MEMORY
RAG
DATABASE/API

Items:

Current order number
Current order status
Return policy
Preferred language
Conversation messages
Refund approval
Current inventory
Customer's preferred communication channel

123. Expected Classification

Information	Best Home
Current order ID being discussed	State/session
Current order status	Order API
Return policy	RAG
Preferred language	User profile/memory
Conversation messages	Session
Refund approval	Workflow/database
Current inventory	Inventory API
Communication preference	Profile/memory

124. Homework Assignment 5 — Memory Risk Audit

Create five examples of dangerous memory behavior.

Example:

Memory Problem:
Stored old order status.

Risk:
Agent provides outdated shipping information.

Solution:
Store order ID, retrieve live order status.

125. Homework Assignment 6 — Conversation Summary

Create a 20-message sample conversation.

Then compress it into:

Customer Goal
Important Identifiers
Current Issue
Actions Already Taken
Open Questions

Compare:

Full Transcript
vs
Structured Summary

126. Homework Assignment 7 — Design a Production Session ID

Instead of:

```
session1
```

design:

```
support:{customer_id}:{case_id}
```

Example:

```
support:C1002:TICKET-883
```

Explain why this is safer and easier to debug.

127. Homework Assignment 8 — Build Memory Policy

Define:

Store

```
Preferred language  
Communication preference
```

Do Not Store in conversational memory

```
Passwords  
API keys  
Payment credentials  
Live inventory  
Dynamic order status
```

Refresh from trusted systems

```
Account balance  
Order status  
Inventory  
Tracking status
```

128. Interview Questions

What is the difference between context and memory?

Context is information currently available to the model; memory is information stored for later reuse.

What is a session?

A session groups interaction history so a later turn can continue the same conversation.

What is state?

Structured information describing current workflow progress.

What is `previous_response_id`?

A Responses API mechanism for continuing from a previous response.

What is `SQLiteSession`?

An Agents SDK session implementation that can persist conversation history under a session ID.

What is local run context?

Developer-defined data passed to application code/tools during an Agents SDK run; it is not automatically included in the LLM input.

What is stale memory?

Previously stored information that may no longer represent the current truth.

Why shouldn't inventory be long-term conversational memory?

Because inventory changes and should be retrieved from the live inventory source.

What is the difference between memory and RAG?

Memory usually stores relevant interaction/user information, while RAG retrieves relevant knowledge from documents or knowledge sources.

Why use structured state?

It gives applications explicit workflow information without requiring the model to repeatedly infer everything from a transcript.

129. Outsourcing Job Perspective

Suppose a client says:

"I need my AI agent to remember customers."

A beginner may respond:

"We'll save every chat."

A stronger AI Automation Architect asks:

What should the agent remember?

For how long?

Is that information stable?

Where is the source of truth?

Should it be user-level or conversation-level?

Can customers have multiple cases?

What happens when stored data becomes outdated?

What information is sensitive?

Should this be conversation memory, CRM data, RAG, or live API data?

That is a far more professional conversation.

130. Example Client Architecture

Client wants an ecommerce support agent.

Design:

```
Customer Login
  ↓
Authenticated Customer ID
  ↓
Support Session
  ↓
AI Agent
├─ Conversation History
├─ Active Ticket State
├─ Customer Preference Memory
├─ Order API
├─ Tracking API
└─ Policy RAG
```

Now each information source has a clear job.

131. Production Memory Design Principle

Avoid:

```
Everything
→ LLM Memory
```

Prefer:

```
Conversation
→ Session

Workflow Progress
→ State

Stable Preference
→ Profile / Long-Term Memory

Company Knowledge
→ RAG

Live Business Facts
→ API / Database
```

132. Day 11 Learning Outcome Checklist

Students should now be able to:

- ☐ Explain context
- ☐ Explain conversation history
- ☐ Explain context windows
- ☐ Explain stateless systems
- ☐ Explain stateful systems
- ☐ Explain sessions
- ☐ Explain short-term memory
- ☐ Explain long-term memory
- ☐ Distinguish state from history
- ☐ Distinguish memory from database records
- ☐ Distinguish memory from RAG
- ☐ Use `previous_response_id` conceptually
- ☐ Understand OpenAI Conversations
- ☐ Understand Agents SDK Sessions
- ☐ Use `SQLiteSession`
- ☐ Explain local application context
- ☐ Explain `RunContextWrapper`
- ☐ Explain run state
- ☐ Identify stale-memory risk
- ☐ Identify dynamic facts that should be refreshed
- ☐ Design session IDs
- ☐ Create memory retention rules
- ☐ Build a multi-turn support-agent architecture

If students can explain why:

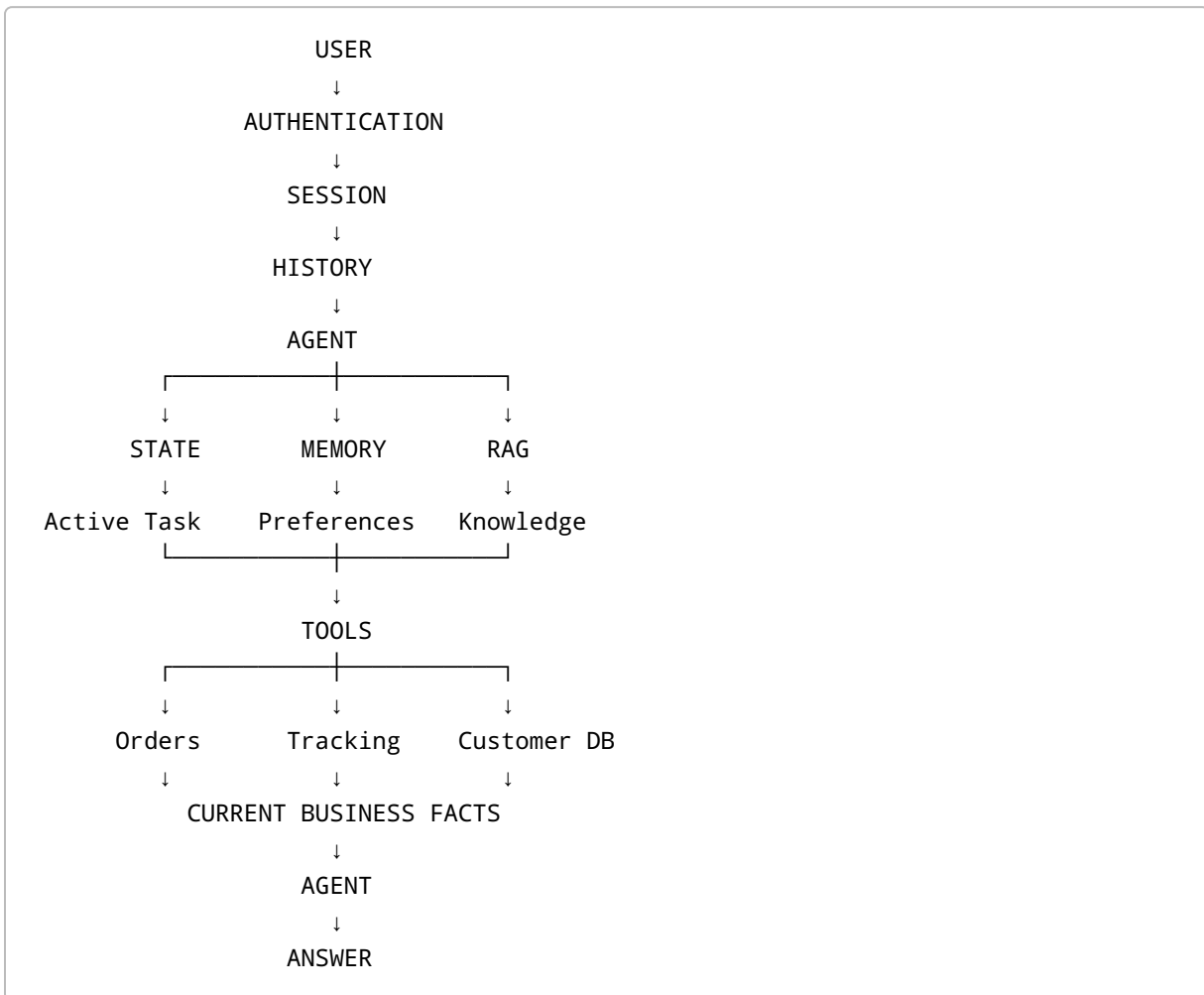
Order ID
can be remembered

but:

Current Order Status
should usually be retrieved again

then they understand one of the most important Day 11 principles.

133. Day 11 Final Mental Model



134. Day 11 Final Takeaway

Students should remember:

Memory should help an agent maintain continuity, but memory should never replace authoritative business data.

The most useful formula is:

Session
=
What happened in this conversation

State
=
Where the workflow is now

Memory
=
Useful information retained for later

RAG
=
Relevant organizational knowledge

API / Database
=
Current business truth

A production-quality agent combines these deliberately instead of treating them as one giant "memory."

Preview — Day 12

Day 12 — Agent Decision Making, Planning, and Control Loops

Day 12 moves from:

What does the agent remember?

to:

How does the agent decide what to do next?

Students will learn:

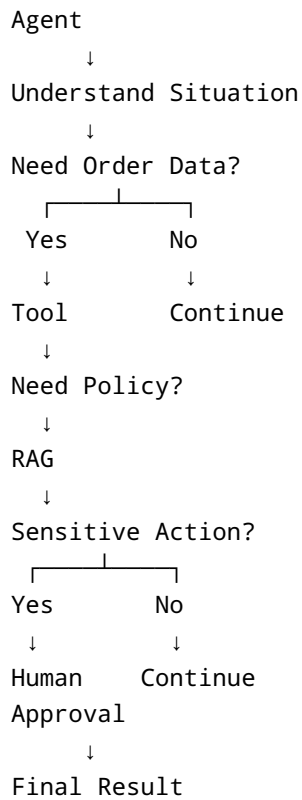
```
Observe
↓
Understand Goal
↓
Check State
↓
Identify Missing Information
↓
Choose Action
↓
Choose Tool
↓
Evaluate Result
↓
Continue / Stop / Escalate
```

Day 12 will cover:

- Agent decision loops
- Goals
- Planning
- Task decomposition
- Tool selection
- Sequential actions
- Parallel actions
- Decision boundaries
- Stop conditions
- Retry decisions
- Escalation
- Confidence
- Deterministic rules vs model decisions
- Approval gates
- Loop limits
- Cost controls
- Failure recovery
- Designing agent policies
- Planning vs execution
- Building a customer-support decision engine

The Day 12 mini project will become:

```
Customer Goal
↓
```



This will take students from a **stateful conversational agent** to an agent that can **systematically decide, act, evaluate, and stop**.